

Towards AI-Powered Automatic 3D Scene Generation for the Metaverse: A Comparative Analysis of Manual and Photogrammetry Techniques

Viviana Pentangelo^{1*}, Dario Di Dario¹, Vincenzo De Martino²,
Marco Dello Buono¹, Stefano Lambiase³

¹Software Engineering (SeSa) Lab, Department of Computer Science,
University of Salerno, Via Giovanni Paolo II 132, Salerno, Italy.

²Department of Service and Information System Engineering (ESSI),
Universitat Politècnica de Catalunya, UPC-Campus Nord, c/Jordi
Girona 1-3, Barcelona, Spain.

³Department of Computer Science, Aalborg University Copenhagen, A.
C. Meyers Vænge 15, Copenhagen, Denmark.

*Corresponding author(s). E-mail(s): vpentangelo@unisa.it;

Contributing authors: ddidario@unisa.it; vincenzo.de.martino@upc.edu;
m.dellobuono1@studenti.unisa.it; stla@cs.aau.dk;

Abstract

The metaverse has recently gained traction as a persistent, realistic, and interconnected virtual-physical ecosystem with high levels of realism and immersivity. According to such a definition, it fundamentally relies on the generation of high-fidelity 3D scenes. Manual construction of these scenes remains a time-consuming process that requires advanced technical skills, posing a significant barrier to widespread adoption. Recent advancements in AI-assisted photogrammetry tools offer an alternative approach by enabling semi-automatic 3D reconstruction from real-world environments. Nevertheless, the usability, quality, and real-time applicability of the resulting assets remain underexplored. This study aims to assess the effectiveness of photogrammetry-based tools for creating functional and visually appealing 3D scenes for metaverse applications. To this end, a comparative experiment was conducted by reconstructing three environments using

manual modeling and two photogrammetry tools based on distinct technologies: Polycam (MVS-based) and LumaAI (NeRF-based). The resulting models were evaluated using quantitative metrics, including modeling time, polygon count, visual similarity (SSIM), real-time performance (GPU usage and FPS), and Hausdorff distance. The findings revealed that photogrammetry significantly accelerates the modeling process but does not consistently outperform manual modeling in terms of polygonal optimization and real-time rendering efficiency. Furthermore, the choice between photogrammetry tools depends on object characteristics and application constraints. This study provides practical insights and empirical guidelines for developers and researchers, highlighting the trade-offs between automation, performance, and visual fidelity in 3D scene generation for the metaverse.

Keywords: Metaverse, Photogrammetry, 3D Scene Generation, Virtual Environments

1 Introduction

In recent years, the concept of the metaverse has evolved from a science fiction term [1] to a tangible and rapidly developing reality. Commonly defined as a persistent, immersive 3D environment where users can interact with digital content and each other in real time [2], where the user experience is envisioned as realistic, believable, and highly immersive [3, 4], the metaverse has attracted growing interest across multiple research domains—including education, simulation, and social interaction [5–8]. Its potential lies in enabling realistic, immersive experiences within controlled and customizable virtual environments.

At the core of metaverse applications are realistic and immersive 3D scenes—i.e., virtual environments that resemble credible settings and allow users to explore, navigate, and interact freely within them [4, 5]. For such environments, a high degree of geometric and visual realism represent a critical aspect, as it directly underpins the credibility and operational validity of advanced immersive applications such as training and simulation [9, 10] or remote workspace orchestration [11]. Replicating credible geometries and materials can be therefore crucial when recreating realistic metaverse scenes. Such scenes are essentially composed of a set of diverse 3D models, which represent an object as a set of vertices in a three-dimensional space; when these vertices are connected through edges and faces and properly textured, they form the shape and appearance of the intended object [12]. Achieving a realistic and credible 3D object inherently involves a trade-off: the more complex and detailed a model is, the more visually realistic it becomes and the better it captures fine-grained characteristics; however, such complexity, mainly related to the number of polygons composing the object and the texture size, also demands greater computational resources to ensure smooth rendering in real-time applications [12, 13]. Consequently, 3D content creators must constantly balance visual fidelity with performance optimization, adjusting the detail level to meet aesthetic and technical requirements.

For the reasons mentioned above, creating 3D scenes that are both visually appealing and high-performing in real-time can be particularly challenging. Moreover, the

creation of 3D models generally demands advanced technical and artistic competencies, which require substantial time and experience to acquire [14, 15]. Such skills are not typically widespread within software development environments. This represents a significant barrier to the development and diffusion of the metaverse, as it discourages practitioners from engaging in the design and implementation of custom virtual environments. As a result, there is a frequent reliance on pre-made assets, which constrains both creativity and opportunities for personalization [15]. The demanding expertise required for 3D modeling, combined with the growing need for readily available 3D assets to experiment with realistic virtual scenes, has opened the space for research efforts aimed at supporting professionals and researchers in the generation and integration of 3D models within metaverse environments.

Artificial Intelligence (AI) has emerged as a promising means to mitigate these challenges. In particular, recent advancements have introduced novel approaches that leverage AI to automate and accelerate the 3D content creation process, thereby reducing the reliance on specialized expertise. One such approach is **photogrammetry**, which involves capturing multiple photographs or videos of a real-world object or environment from various angles and algorithmically reconstructing a 3D model that replicates the original geometry and texture [16, 17]. Photogrammetry technologies mainly follow two approaches: (1) Multi-View Stereo (MVS), which reconstructs 3D geometry by matching features across overlapping images [18], and (2) Neural Radiance Fields (NeRF), which uses neural networks to generate photorealistic views by learning the color and density of points in space [19, 20]. Building upon these technologies, some tools have been proposed to make them accessible to a broader audience, including Polycam [21], which leverages MVS-based pipelines, and LumaAI [22], which implements a NeRF-based approach. These tools hold great value as they wrap complex computational pipelines within user-friendly interfaces, enabling even users with limited 3D modeling or AI expertise to create realistic digital assets with minimal effort.

Despite the growing availability of tools that leverage photogrammetry for 3D reconstruction, their adoption has not yet become standard practice in the development of virtual environments. A key limitation lies in the uncertainty regarding the quality and usability of the generated outputs—particularly in terms of balance between visual complexity and rendering performance, which is critical in real-time applications [13, 23]. Although prior research has investigated the use of photogrammetry to reconstruct complex environments and avatars [24–26], **there is a lack of empirical studies evaluating whether the photogrammetry resulting assets are ready for practical use in metaverse applications**. Specifically, there is limited understanding of whether such tools can effectively *democratize* 3D scene creation, enabling users without prior modeling expertise to produce usable virtual environments.

This study aims to contribute to the body of knowledge in the field of automated 3D asset creation by providing practical insight into the use of photogrammetry-based tools to create visually appealing and functionally optimized 3D scenes for the metaverse. In our previous conference paper [27], we conducted a first comparative experiment in which the same indoor scene was recreated both manually and using the

Polycam tool. In this study, we present an extension of our conference contribution: we build on our preliminary results and expand the scope to enhance the robustness and generalizability of the findings.

© Goal of the study

Conduct a comparative experiment involving a manual modeling technique and photogrammetry-based tools to assess the usability of such tools for creating realistic and functional 3D scenes for the metaverse.

Specifically, we reconstructed a total of three scenes and a broader comparison, not only between manual and automated approaches, but also between two photogrammetry-based tools based on different underlying technologies—Polycam based on MVS and LumaAI based on NeRF, respectively. The reconstructed scenes were then evaluated using a set of quantitative metrics, each capturing a key dimension relevant to 3D scene creation for metaverse applications. These included: completion time (technique efficiency), polygon count (model complexity), Structural Similarity Index (SSIM) (visual realism), GPU usage and average FPS (real-time performance), and Hausdorff distance between corresponding models generated by Polycam and LumaAI (geometric divergence across tools).

Our results provide three main contributions. First, it presents a comparative evaluation of manual modeling and two mobile photogrammetry workflows of 3D scenes tailored to metaverse applications. Second, it highlights the primary trade-offs observed among visual quality, modeling time, and real-time performance across the different reconstruction techniques. Finally, it offers empirically grounded insights intended to support both practitioners and researchers in selecting the most suitable workflow approach (automation tools versus manual modeling) based on specific project requirements and application contexts.

The paper is structured as follows. Section 2 provides a theoretical background and the work related to our study. Section 3 reports in detail the experiment design and procedure that we followed. Section 4 outlines the obtained results based on the computed evaluation metrics. Section 5 provides the discussion points from our results and some practical empirical guidelines. Finally, Section 6 draws the conclusion and future research directions.

2 Background & Related Work

This section reports on the background and related work that are foundation of our contribution.

2.1 The Metaverse and the Importance of Realism

According to the literature, the metaverse has been envisioned as a three-dimensional online environment where users, represented by customizable digital avatars, interact in persistent and interconnected virtual spaces [28]. More generically, the metaverse

is a broad concept that is historically rooted in early virtual environments and simulation designs [3], which aims to become a persistent virtual-physical ecosystem [9]. In this vision, its main features encompass ongoing actions, lifelike experiences, interoperability, and architectures that can scale [4, 29]. This environment functions in real-time, enabling thousands or even millions of users to interact concurrently, while achieving a high degree of immersivity, and realism [4].

Researchers and practitioners also envision the metaverse as a broader virtual ecosystem which is not completely disconnected from the real world, but has an intersection with the physical reality [29]. As an example, the metaverse is often connected to the concept of digital twins, which serve as synchronized 3D replicas of physical entities or entire environments [30]. Connecting the virtual to actual physical scenes or environments is crucial for metaverse systems requiring high fidelity and credibility, such as medical training, surgical simulation, and remote workspace orchestration [9, 10]. In all of these kind of applications, user may want to have faithful representation of their environments within a safe virtual simulation. Consequently, achieving high fidelity environments by replicating the exact geometries, lighting behaviors, and physical imperfections of a real-world room, directly influence the credibility of these immersive infrastructures [10].

2.2 Reconstruction Techniques fo 3D Scene Generation

The task of building highly realistic virtual environments which are also faithful to real ones is complex and requires a mix of diverse technical skills. Their design typically involves professionals with diverse expertise, including artists and software developers. Conventional approaches to build 3D scenes typically rely on manual modeling techniques that require proficiency in specialized software [31], where object meshes must be recreated through the manipulation of vertices and polygons [15]. Given the substantial effort involved in crafting these environments, research has started to exploit the automation power offered by computer vision techniques, which seek to automate the creation of 3D content, thereby reducing the need for technical modeling skills.

One of the most popular techniques is **photogrammetry**, an established reconstruction technique that extracts the 3D geometric properties of physical objects or environments from a sequence of overlapping two-dimensional photographic images [32]. Unlike active scanning methods such as LiDAR, photogrammetry relies entirely on ambient illumination to detect features, surface textures, or target markers across adjacent frames to infer depth and scale. Within this domain, Multi-View Stereo (MVS) photogrammetry extends traditional stereophotogrammetry by leveraging highly redundant image sets captured from multiple spatial perspectives. The standard MVS computational pipeline executes feature detection and matching across overlapping views to resolve camera positions and subsequently applying geometric triangulation to generate explicit 3D point clouds and solid polygonal meshes [33].

Another emerging recostruction method is the family of the Neural Radiance Fields (**NeRF**) techniques, which are methods for reconstructing a continuous volumetric scene function from a sparse set of input views [19]. In contrast with MVS, NeRF techniques are deep learning based, since the scene is encoded within the weights of a fully connected multilayer perceptron network, that maps the continuous coordinates

directly to its corresponding volume density and view-dependent emitted RGB radiance. To overcome the inherent constraints of the original architecture, research has introduced numerous extensions of the original NeRF implementation that made these set of technique more and more optimized, performing and subsequently adopted [20].

The study of these techniques for reconstructing 3D scenes has begun to interest experiments in the literature. Abramov et al. [34] employed drone-based photogrammetry to generate detailed 3D representations of rural settlements, optimized for integration into metaverse platforms such as Voxels and Mona, with the aim of supporting rural revitalization and digital economies. Prasetyadi et al. [26] utilized terrestrial photogrammetry tools—such as Polycam—to create realistic avatars and immersive environments for the Science and Technology Region (KST) within the metaverse. Additionally, Gledhill et al. [25] proposed a methodology for optimizing photogrammetric data for metaverse applications, focusing on mesh and texture enhancements to ensure realism and performance on low-end VR devices. Other related work specifically investigated the performance and usability of MVS and NeRF techniques to reconstruct 3D environments. Murtiyoso et al. [35] investigated the geometric quality of implicit neural point clouds within indoor environments by benchmarking NeRF models against commercial and open-source MVS algorithms. Their analysis revealed that while NeRF offers competitive training speeds and can successfully reconstruct challenging textureless or reflective materials where standard MVS approaches fail, the extracted point clouds exhibit significant noise levels. To systematically evaluate these geometric limitations, Remondino et al. [20] performed a comprehensive metrological appraisal comparing various NeRF configurations against traditional photogrammetry across multiple material profiles, including metallic, translucent, and transparent objects. Their quantitative assessment confirmed that neural implicit models like Instant-NGP and Nerfacto exhibit superior performance when reconstructing refractive or highly specular surfaces, whereas conventional photogrammetry continues to provide cleaner, less noisy models over objects well-defined surface textures. Furthermore, Croce et al. [36] extended this comparative paradigm across multiple geometric scales, examining the specific impact of data minimization and spatial downscaling on both volumetric models and classical discrete meshes. Their results indicated that under conditions of reduced input data or lower resolution, NeRF outperforms photogrammetry in preserving completeness and material description for the same set of input images.

As outlined above, significant research has been dedicated to comparing 3D reconstruction techniques across various domains and validating their reliability against manual measurements. However, limited attention has been given to the evaluation of such techniques in terms of efficiency and usability for metaverse development, where real-time rendering and immersive realism are essential. To date, no studies have systematically analyzed the strengths and limitations of photogrammetry in contrast with manual methods for the creation of real-time 3D environments in the metaverse.

To address this gap, a first comparative study was previously conducted in which an identical indoor environment was reconstructed both manually and through the Polycam photogrammetry tool [27]. Building upon these preliminary findings, the present work expands the investigation to improve the robustness and generalizability

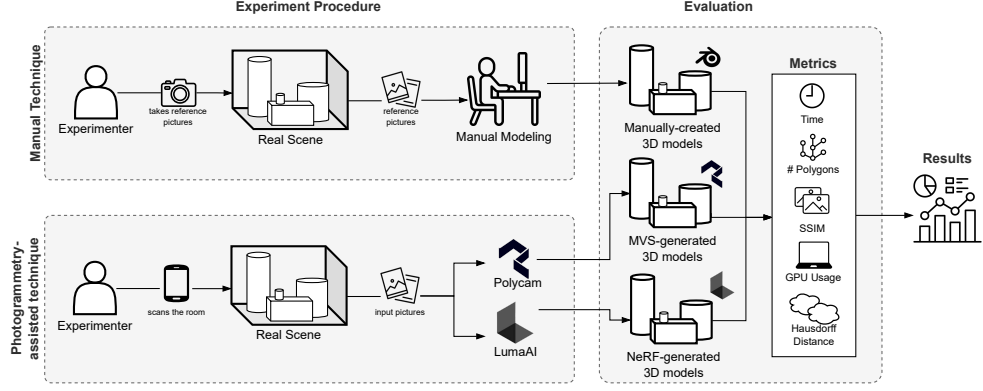


Fig. 1 Overview of the experiment’s procedure and evaluation

of the results and to generate actionable insights for metaverse-oriented development workflows.

3 Experiment Design

To achieve our goal, we designed and conducted a comparative experiment involving two different techniques for 3D scene creation: manual modeling and photogrammetry-assisted modeling using freely accessible tools. The objective was to investigate the extent to which automated techniques for generating realistic 3D models can be effectively used to recreate virtual environments for the metaverse, specifically focusing on freely accessible mobile tools that serve as wrappers for such reconstruction algorithms, so to abstract the user from specific technical skills. Additionally, the experiment compares two photogrammetry tools—Polycam, which relies on MVS reconstruction, and LumaAI, which employs Neural Radiance the deep learning-based NeRF. The choice for the selection of such tools was driven by our focus on ready-to-use mobile wrapped tool. We extended our previous comparison [27] with a tool which implemented a different reconstruction approach, namely LumaAI using NeRF. We also selected reconstruction methods that output a 3D polygonal mesh, which led us to select the MVS modality of Polycam instead of the 3DGS one. This was due the point-based rasterization given by the latter [37], that is incompatible with standard real-time game engine and the metrics selected for our study. Figure 1 summarizes the overall experiment’s design and procedure.

1. First, we selected three scenes representing typical life scenarios and took photos of the objects inside them (for the manual modelling) and scanned them using a camera (for the photogrammetry-based approach). Our choice was guided by the emphasis of placed by Dionisio et al. [4] on realism, social presence, and interactivity as defining characteristics of the metaverse. Based on this, we chose typical environments that metaverse developers may wish to incorporate into their platforms: a domestic indoor environment (Scene A—a living room), a working environment

- (Scene ②—an office), and a social environment (Scene ③—a meeting hall). Each scene comprised multiple objects varying in size, shape, and texture.
2. Then, one of the authors—with expertise in 3D modeling and texturing, hereafter referred to as the “experimenter”—was designated as the main participant responsible for creating all scenes using manual and photogrammetry-assisted techniques.¹
 3. Lastly, the generated 3D models were integrated into scenes designed to resemble metaverse environments. A set of evaluation metrics (described in Section 3.3) was computed, and the results were compared across the different setups to derive insights and formulate preliminary guidelines for practitioners.

3.1 Employed Techniques

In this study, we employed two major techniques to conduct the same task of obtaining a 3D representation of a real scene. The second technique will be divided into the use of two different tools, obtaining a total of three representations for each scene.

- ✂ **Manual Modeling.** This technique involves manually modeling each object by defining its geometry through the manipulation of vertices, edges, and faces within a 3D modeling environment. The process is eventually completed with a texturing process to resemble the real materials of the actual objects. We adopted Blender² for this purpose, as it is a widely accessible open-source tool with extensive documentation and community support. For each scene, the experimenter reconstructed all the objects from scratch, creating and applying textures through the material node system and relying on photographic references of the real-world counterparts.
- ✂ **Photogrammetry.** This technique utilizes photogrammetry-based mobile tools to scan real-world objects using a smartphone camera and automatically generate 3D reconstructions. Two applications, each based on a distinct reconstruction algorithm, were selected to explore the usability of such tools in different contexts. Both tools are freely available and compatible with standard mobile devices, making them accessible options for metaverse developers without specialized hardware or advanced 3D modeling expertise.
 - **Polycam.** This tool reconstructs 3D models by processing a series of overlapping images of a real-world object or environment captured from different angles using a mobile device. It uses an MVS pipeline [18], which first extracts features from each image and matches them across views to estimate the relative positions of the camera. Then, it triangulates the matched features to generate a dense point cloud, which is then converted into a mesh. Textures are generated by projecting the original photos onto the mesh surface, resulting in a photorealistic appearance. The entire process is automated and optimized for mobile performance [21].

¹The manual creation for Scene ① and its reconstruction with Polycam has been leveraged from the results of our preliminary experiment [27]. We integrated and completed such results with its LumaAI reconstruction and the computation of the new metrics. Scene ② and Scene ③ were modeled and reconstructed from scratch, and the experimenter ensured to follow the same manual workflow previously used for Scene ①.

²<https://www.blender.org/>

- **LumaAI.** This tool leverages NeRF-based models for the reconstruction [19]. Unlike traditional photogrammetry, which relies on geometry-based triangulation, NeRF uses a neural network to learn the volumetric radiance field of a scene from a set of 2D images taken from different viewpoints. The network learns how light behaves within a scene, mapping each 3D point and viewing direction to a corresponding color and density value. This approach enables detailed and photorealistic reconstructions, which are particularly effective for capturing complex lighting and surface properties.

Both tools were selected because they allow 3D reconstruction using a mobile phone and can process the same input data—photos or videos of real-world objects. However, each tool relies on a different underlying reconstruction technique. This setup enables a direct comparison of the output quality and usability, allowing us to investigate whether and how the reconstruction method affects the final results and to identify any significant differences between the two approaches.

3.2 Experiment Procedure

To guide the experiment’s design, we conducted a pilot study. One co-author tested the procedure in an indoor environment, which was not part of the three planned scenes for the experiment (a domestic kitchen), before conducting the main experiment. The aim was to gain hands-on familiarity with the selected tools and modeling techniques and identify potential challenges in their use. This preliminary exploration also served to fine-tune the experimental procedures, which were then applied consistently in the main study, as described in the following section.

The experiment’s procedure began with setting up the scene. The authors carefully prepared each scene to include a variety of objects representative of the target environment, ensuring a range of shapes, sizes, and textures. For example, the living room (Scene Ⓐ) included items such as a coffee table, plants, and decorative elements, while the office (Scene Ⓑ) featured desks, chairs, documents, and stationery. A complete list of the objects is shown in Table 1. The experimenter accessed each environment twice—once for each technique. For the manual modeling, the experimenter was first allowed to take pictures of the room and its objects to be used as visual references during the modeling process. The experimenter independently determined the number and type of reference images needed for accurate modeling and texturing. The time taken to capture these references was included in the overall time measurement for the manual modeling process. Following this step, the experimenter recreated the entire scene using Blender. Modeling sessions were limited to a maximum of two hours per session and repeated over multiple days until the scene, including geometry and texturing, was fully completed. For the photogrammetry-based approach, the experimenter revisited the room and used a smartphone to capture the scene by photographing each object from multiple angles. The selected tools—Polycam and LumaAI—processed these images to generate textured 3D models.³ Blender was subsequently used only to assemble the reconstructed assets into a coherent scene layout.

³The living room was subject to our first experiment [27], for which we will consider our preliminary outcomes in our results. Since the second tool was not present in the first work—i.e., LumaAI—the same pictures used on such a tool to generate the remaining models.

Table 1 Overview of the objects of each scene to be reconstructed.

Scene	Objects
Scene (A)	Clock
	Painting
	Wardrobe
	Chimney
	Lamp
	Plant
	Sofa
	Curtains
	Air Conditioner
	Coffee Table
	Heater
Scene (B)	Whiteboard
	Printer
	Big Desk
	Office Chair
	Work Desk
	Keyboard
Scene (C)	Telephone
	Chair
	Big Whiteboard
	Cabinet
	Small Whiteboard
	Meeting Table

Three distinct 3D reconstructions were obtained for each scene—one for each modeling technique and tool. When the scenes were completed and assembled, Blender was used to extract quantitative metrics related to model complexity and level of detail and to generate renders with lighting settings similar to the original scenes. Subsequently, the experiment procedure’s last phase consisted of exporting all reconstructed scenes to Unity3D,⁴ a widely adopted game development engine, to assess their functional performance. This phase was essential to evaluate how each model behaved in a real-time environment, particularly in terms of rendering efficiency and interactivity. We created a scene for each reconstruction to obtain these insights and ran a real-time exploration with a first-person camera and free 3D movement for 5 minutes each.

3.3 Metrics for Evaluation

To obtain and evaluate the experiment’s results, we computed a set of metrics. Each of the chosen metrics, explained in the following, aimed to capture insights into the different aspects of the evaluation process.

- **Time for Completion.** This metric was employed to obtain insights on the *techniques efficiency*. For each technique, the total time required to complete the 3D reconstruction was measured in hours and minutes, starting from the experimenter’s

⁴<https://unity.com/>

initial access to the room and ending with the completion of the final scene. This time tracking was intended to assess how much photogrammetry tools can significantly accelerate the 3D scene creation process compared to manual modeling.

- ▶ **Number of Polygons.** This metric was employed to obtain insights on the *3D models' complexity*. For each model of each scene, we measured the number of polygons, specifically triangular faces, that compose each mesh. A higher polygon count typically corresponds to greater visual detail, which can enhance realism but may also impact performance, particularly in real-time applications: higher polygon counts increase the demand for hardware and software resources to ensure smooth simulation [13]. This aspect is especially relevant in the context of the metaverse, where real-time rendering is essential for interactive and immersive experiences [4]. We note that the experimenter was not instructed to match a specific polygon count during manual modeling. In both approaches, the shared objective was to achieve a result that closely resembled the real-world reference from a visual standpoint.
- ▶ **Structural Similarity Index (SSIM).** This metric was employed to obtain insights on the *3D reconstruction realism*. The SSIM metric [38] compares two images based on three key components: luminance, contrast, and structural information. It ranges from -1 (maximum dissimilarity) to 1 (maximum similarity). We employed this metric to assess the visual similarity between photographs of the actual living room and the corresponding 3D reconstructions. To compute it, we first took three photos of each real room from different viewpoints, ensuring that all objects appeared in at least one image. We then generated three corresponding renders from the 3D scenes in Blender, carefully aligning the virtual camera angles to match the real photographs and creating a similar lightning setting, simulating the act of taking the same shots within the virtual environment. SSIM was then calculated for each pair of real and rendered images. We opted to apply this method to the entire room scene rather than to individual objects, as our goal was to evaluate the overall realism of the reconstructed environment rather than the fidelity of single elements.
- ▶ **GPU Usage & Mean Frames Per Second (FPS).** These metrics were employed to obtain insights on the *scenes' functionality for real time rendering*. We measured the GPU usage required to run the real-time simulation of each reconstructed scene. To obtain this measurement, all reconstructed scenes were exported to Unity3D; we configured the game scene with a first-person controller, allowing free movement via mouse and keyboard input. The simulation was executed for five minutes on three different machines, each equipped with a distinct NVIDIA RTX GPU with from three different generations based on our availabilities: a second generation GPU (RTX 2080), a third generation GPU (RTX 3060), and a fourth generation GPU (RTX 4090). We recorded the average GPU usage percentage and average FPS during each session to assess the rendering performance across different hardware profiles.
- ▶ **Hausdorff Distance.** This metric was employed to obtain insights on the *differences among the 3D models*. It quantifies the geometric dissimilarity between two 3D shapes by measuring the maximum distance from a point on one mesh to the closest point on the other. A higher distance indicates greater dissimilarity between

the models [39]. This metric was selected to evaluate differences between the generated and manually created models and between the generated models. In particular, it allowed us to investigate whether the two photogrammetry tools produced significantly different results when given the same input images. We computed the Hausdorff Distance and analyzed the resulting values for each pair of models representing the same object (manual vs. Polycam, manual vs. LumaAI, and LumaAI vs. Polycam).

3.4 Threats to Validity

This section presents the threat and limitations emerging from our research design.

A potential threat to internal validity arises from the dependence on a single participant’s proficiency and familiarity with the tools employed in the experiment, specifically Blender, Polycam, and LumaAI. This reliance introduces concerns regarding the reproducibility of the results, as outcomes may vary with participants possessing different levels of expertise. For instance, a greater familiarity with Blender than with Polycam could introduce an implicit bias favoring manual modeling techniques over automated photogrammetric generation. Moreover, consistent task execution across both techniques is critical. In this study, the same co-author performed all modeling tasks using both methods, potentially introducing personal biases or methodological inconsistencies. These factors may influence the comparative results, not due to objective differences in performance, but rather due to subjective familiarity with the tools. However, given that this study aims to explore and demonstrate the potential of an automated modeling approach rather than to establish its definitive superiority, this decision was considered an acceptable methodological trade-off.

The restricted number and localized nature of the evaluated environments inherently limit the generalizability of our quantitative metrics. Because this study explicitly targets the end-user mobile capture workflow to evaluate how easily non-experts can digitize spaces at their immediate disposal, our protocol strictly requires handheld interaction within an accessible setting to measure real-world application usability. We had to exclude considering the use of existing large public datasets, since recreating large-scale benchmark datasets manually in Blender to establish a comparative baseline would be logistically unfeasible and outside our operational scope. To mitigate this limitation, the original single-scene framework [27] was expanded by integrating two new structurally distinct real-world environments. Nevertheless, evaluating three scenes remains a constrained sample size from a large-scale data benchmarking perspective. Therefore, our findings should be interpreted specifically as practical and exploratory insights into real-world practitioner workflows and mobile tool usability. Future expansions should aim to progressively enlarge the set of evaluated environments and involve a larger cohort of modelers.

A threat to construct validity stems from the absence of an absolute physical 3D ground truth reference, such as a high-fidelity dataset captured via an active terrestrial laser scanner. Although the manually modeled Blender meshes provide a clean baseline for scene layout comparison, utilizing them as a strict geometric ground truth for absolute error calculation would not have been methodologically sound, due to the radical topological differences between human-crafted, low-polygon structures and automated



Fig. 2 Overview of the obtained scenes.

dense reconstructions. Applying traditional vertex-to-vertex validation metrics across these fundamentally distinct mesh architectures would yield mathematically skewed and skewed results that evaluate human design optimization choices rather than actual spatial reconstruction error. Consequently, this empirical evaluation is structurally constrained to assessing relative tool performance and visual alignment rather than absolute accuracy.

Another potential limitation pertains to the adequacy of the selected metrics in capturing the intended constructs. The evaluation relied on measures such as modeling time, polygon count, Structural Similarity Index Measure (SSIM), and GPU usage to assess quality and efficiency. Although these indicators provide useful quantitative insights, they may not fully represent all dimensions of practical relevance, particularly those related to user experience in interactive applications or performance in real-time rendering contexts. For example, a model with a lower polygon count may perform better computationally but could compromise visual realism, thereby diminishing the immersive quality of the experience. As such, an exclusive reliance on these metrics may offer a partial view of the effectiveness and usability of each modeling technique.

4 Analysis of the Results

In the following section, we present the results obtained based on the evaluation metrics.

The experiment produced three distinct 3D representations for each environment, resulting in a total of nine reconstructed scenes—one per technique (Manual, Polycam, and LumaAI) for each setting. The scenes generated using the photogrammetry-based tools were processed and assembled within Blender to ensure consistency in layout and lighting. All the material produced during the study, including raw assets, models, and comparative visualizations, is publicly available in the online appendix [40]. Figure 2 shows an overview of the obtained scenes for each technique.

4.1 Time for Completion

As an initial insight, we measured the time required to complete each scene to evaluate the efficiency of each modeling technique. Specifically, the completion time was computed as the total duration from the initial data acquisition phase (manual reference photo collection or environment scanning) to the final 3D scene assembly in Blender. Since the two tools received input from the same videos, their completion time is summarized in a single measure. Table 2 reports the results for all nine scenes, organized by technique.

Table 2 Completion time for each scene.

Scene	Manual	Polycam & LumaAI
Scene (A)	10h 30m	2h 40m
Scene (B)	6h 35m	2h 10m
Scene (C)	5h 10m	1h 50m
Mean	7h 25m	2h 12m

As expected, the manual technique required significantly more time than the automated approaches. On average, scenes modeled manually required approximately 7 hours and 25 minutes, whereas those reconstructed using Polycam and LumaAI required only 2 hours and 12 minutes. These results suggest that **automated reconstruction techniques reduced modeling time by over two-thirds**. The observed variation in completion time across scenes within the same technique appeared to correlate with the number of objects and the overall level of detail in the original environment. For instance, Scene (A) was consistently the most time-consuming to reconstruct, regardless of the technique adopted, due to its higher complexity. This trend remained consistent for both manual modeling and automated photogrammetry-based reconstruction. However, such time differences across scenes for the automated techniques were less pronounced; this may be attributed to the underlying automation and repetitive process, as recording a video of an object is not directly related to its level of detail.

This initial measurement confirmed the notable acceleration achievable through photogrammetry techniques compared to manual modeling. Although automated tools still require time to capture detailed video sequences and to process the data to generate the 3D model, the total time investment remains significantly lower than that required for manual reconstruction, which is still markedly more time-intensive.

4.2 Number of Polygons

To assess the complexity of the resulting 3D models, the total number of polygons was computed for each scene and technique. The results, summarized in Figure 3, highlight considerable differences between the manual and automated approaches.

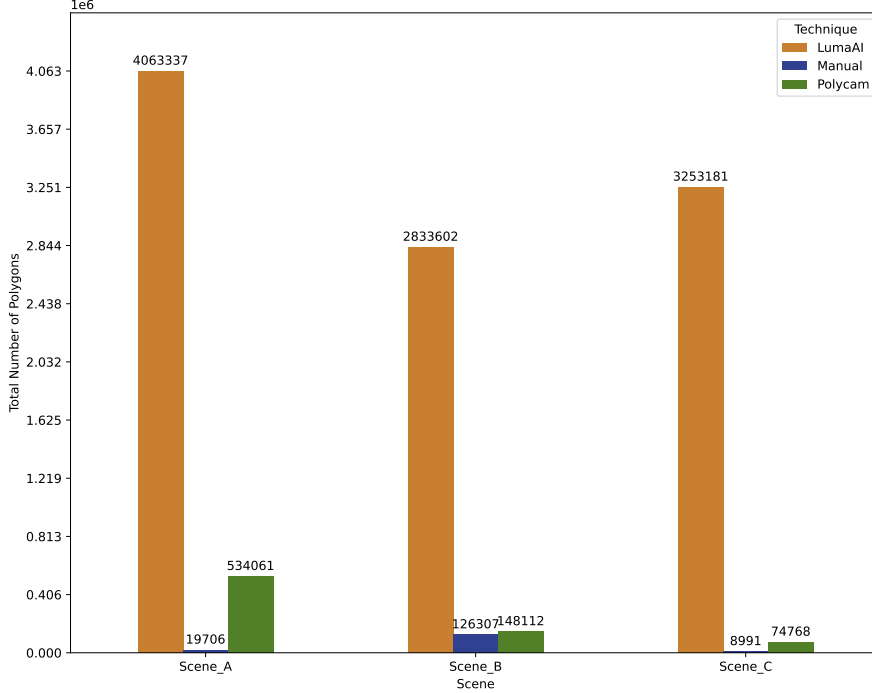


Fig. 3 Total number of polygons for each scene and each technique.

Manual modeling consistently resulted in the lowest polygon counts, with values ranging from only 8,991 in Scene ③ to 126,307 in Scene ②. This outcome reflects the inherently optimized topology achieved through manual mesh construction in Blender, where the experimenter operated in *Edit Mode* to manipulate each vertex and face starting from primitive shapes. This process naturally produces lightweight and uniform geometries, even for complex or irregular objects. As an example, Figure 4 shows the polygon count and topology for the same work chair.

In contrast with the manual results, models generated with LumaAI exhibited extremely dense meshes, with polygon counts exceeding 2.8 million in all scenes and peaking at over 4 million in Scene ①. These results are consistent with the nature of NeRF-based photogrammetry methods, which aim to capture fine surface details through high-resolution reconstruction [20]. While such richness enhances visual realism, it poses challenges for real-time rendering, especially in resource-constrained environments like VR or mobile platforms. Polycam maintained a considerably more moderate polygon count, ranging from approximately 74,000 to 534,000. This balance suggests an internal optimization mechanism that reduces mesh density while preserving essential details. Regarding the two tools, this comparison highlights a key trade-off: Polycam provides a more lightweight output suitable for performance-sensitive environments, yet is more sensitive to finer details and may suffer from geometric incompleteness; LumaAI maximizes fidelity and surface detail, but generates meshes that may be computationally prohibitive without further optimization.



Fig. 4 Comparison of the meshes obtained for the Working Chair object. The manually created chair contains the lesser polygons, resulting in a clearer mesh. The Polycam version increases the count, still within a reasonable range for interactive applications. However, despite its moderate complexity, the mesh includes a topological error—a visible hole in the backrest. In contrast, the LumaAI reconstruction reaches almost 1M polygons, delivering a highly detailed surface with no missing geometry but at the cost of an extremely dense mesh.

4.3 SSIM Values

To assess the visual fidelity of each reconstructed scene, the Structural Similarity Index (SSIM) was computed for three viewpoints (angles) per scene. SSIM compares rendered images of the 3D scene against real photographs of the original environment, quantifying visual similarity on a scale from -1 (no similarity) to 1 (perfect match). Table 3 reports the SSIM values for each technique across all scenes and angles. To obtain visual insights of the results, we generated the SSIM maps by overlapping the two pictures under examination. Figure 5 shows an example of the SSIM maps for Scene (A), Angle 3.

Table 3 SSIM values for each image and technique across the three scenes.

Scene	Angle	Polycam	Manual	LumaAI
Scene (A)	1	0.58297	0.55420	0.44150
	2	0.63305	0.56612	0.43628
	3	0.52478	0.41081	0.41981
Scene (B)	1	0.61454	0.62902	0.60870
	2	0.58918	0.51791	0.54891
	3	0.54835	0.56796	0.53865
Scene (C)	1	0.52454	0.53834	0.49610
	2	0.56766	0.57940	0.54327
	3	0.54811	0.58899	0.54538

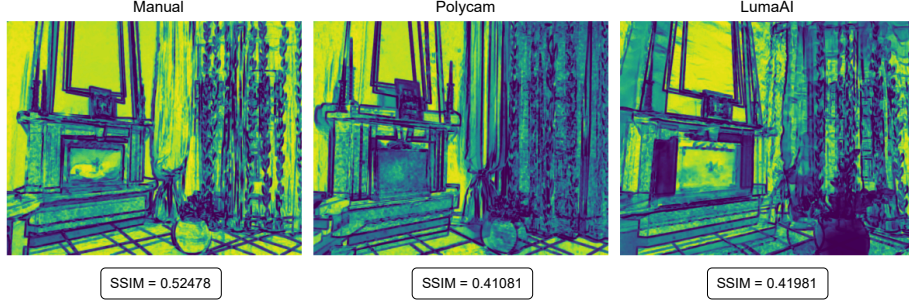


Fig. 5 SSIM maps for Scene A, Angle 3. Higher SSIM values are represented by brighter yellow areas and lower similarity by darker black regions.

The manual technique displays uniform similarity across the entire image, especially on flat surfaces such as the walls and the floor. Polycam reconstruction captures higher similarity around key elements like the fireplace and curtains, though some noise and local inconsistencies are visible. The LumaAI result, while visually rich in geometry, shows more pronounced inconsistencies and a less homogeneous SSIM distribution.

Results indicate that no single technique consistently outperformed the others across all scenarios. In Scene (A), Polycam achieved the highest similarity values overall (avg. SSIM ≈ 0.58), while the manual technique produced slightly lower values and LumaAI consistently performed worst. This trend suggests that in complex, highly furnished scenes such as Scene (A), Polycam’s approach better preserved the original visual appearance, possibly due to its more balanced mesh and texture generation pipeline. Conversely, in Scene (B), both Polycam and the manual technique achieved comparable scores (avg. SSIMs above 0.58), while LumaAI also returned satisfactory results. Here, the difference between techniques was less pronounced. Finally, in Scene (C), the manual technique consistently produced the highest SSIM values across all three angles (avg. SSIM ≈ 0.57), slightly outperforming Polycam and LumaAI. This result may be attributed to the relatively simple object shapes, which made manual modeling and texture mapping more effective.

Overall, the results confirmed that Polycam provided a robust compromise between automation and realism, achieving competitive SSIM scores while maintaining a moderate polygon count. Interestingly, despite LumaAI producing highly detailed and complete meshes, it did not consistently translate this detail into visual similarity with the real scene—suggesting possible limitations in texture generation or lighting reconstruction.

4.4 GPU Usage & Mean FPS

To evaluate the real-time performance of the obtained scene, which is required for metaverse scenes, we imported each of them on Unity and ran a simulation for five minutes. We simulated each scene on three machines with three NVIDIA RTX GPUs. The simulation results further illustrate the trade-offs observed in previous sections. As expected, both the scenes’ polygon count and the hardware’s computational capabilities significantly influenced the performance outcomes. The results of the simulations are summarized in Table 4.

Table 4 Mean GPU usage (%) and FPS during Unity3D simulations for each GPU, scene, and technique.

GPU	Scene	GPU Usage (%)			Mean FPS		
		Manual	Polycam	LumaAI	Manual	Polycam	LumaAI
RTX 2080	Scene (A)	38	43	85	58	55	30
	Scene (B)	36	40	80	59	57	32
	Scene (C)	34	38	82	60	58	31
RTX 3060	Scene (A)	34	39	80	60	60	35
	Scene (B)	33	37	78	60	60	36
	Scene (C)	32	36	76	60	60	37
RTX 4090	Scene (A)	12	15	40	120	115	90
	Scene (B)	10	14	38	120	118	92
	Scene (C)	8	12	35	120	119	95

For the RTX 2080, performance degradation was evident when handling LumaAI models. While Manual and Polycam models maintained acceptable frame rates around 55–58 FPS, LumaAI models caused a sharp drop to approximately 30–32 FPS across scenes. GPU usage also increased notably for LumaAI, reaching values above 80%, reflecting the heavy computational load imposed by the dense and complex meshes. The RTX 3060, a more recent mid-tier GPU, provided a more balanced performance. Manual and Polycam models were handled comfortably at 60 FPS, with GPU usage remaining between 32% and 39%. However, even the RTX 3060 showed signs of strain when rendering LumaAI models, where GPU usage approached 80%, and frame rates dropped to approximately 35–37 FPS, depending on the scene. In contrast, the RTX 4090 easily managed all models with minimal GPU usage and exceptionally high frame rates. Manual and Polycam models were consistently rendered at 115–120 FPS across all scenes, with GPU usage below 15%. Even LumaAI models, despite their extremely high polygon counts, maintained frame rates above 90 FPS, with GPU usage peaking at 40%. Overall, these results confirm that the choice of modeling technique directly impacts performance, particularly in real-time applications. Manual and Polycam models proved to be manageable across a wide range of hardware, while LumaAI models, although visually rich, introduced significant computational overhead. Consequently, unless used in systems equipped with high-end GPUs like the RTX 4090, models generated through LumaAI may require substantial optimization to ensure satisfactory real-time performance.

4.5 Hausdorff Distance

To complement our analysis and further investigate geometric divergence at the object level, we computed the Hausdorff distance between the meshes generated by LumaAI and Polycam for each object. Given the high polygon count observed in the LumaAI models—often several times greater than those of Polycam—we expected substantial geometric differences. The Hausdorff metric enabled us to quantify these differences by

measuring the greatest minimal distance between the two point sets representing each model. It is crucial to emphasize that the Hausdorff metric is deployed here strictly as a comparative indicator to quantify the spatial divergence between the two automated reconstruction paradigms under identical dataset conditions. This comparative metric allows us to evaluate whether choosing one automated pipeline over another under identical environmental constraints yields structurally equivalent or drastically different outputs. Table 5 summarizes the computed Hausdorff distances per object for all three scenes.

Table 5 Hausdorff distance between Polycam and LumaAI models for each object.

Scene	Object	Hausdorff Distance
Scene (A)	Clock	0.3617
	Painting	0.3952
	Wardrobe	1.0182
	Chimney	0.6171
	Lamp	1.6724
	Plant	0.8055
	Sofa	1.5657
	Curtains	1.3527
	Air Conditioner	0.3229
	Coffee Table	0.9791
	Heater	0.8034
Scene (B)	Whiteboard	0.2295
	Printer	0.1013
	Big Desk	0.2801
	Office Chair	0.9089
	Work Desk	0.1296
	Keyboard	0.0719
Scene (C)	Telephone	0.1306
	Chair	0.0570
	Big Whiteboard	0.1715
	Cabinet	0.0987
	Small Whiteboard	0.0797
	Meeting Table	0.2670

The results confirmed our original hypothesis. In Scene (A), the models exhibited the highest discrepancies, with several objects exceeding a distance of 1.0. Notably, the lamp (≈ 1.67), sofa (≈ 1.56), and curtain (≈ 1.35) were among the most divergent, likely due to their complex surfaces and the denser, less regular topology produced by LumaAI. In Scene (B), distances were generally lower, with most objects below 0.3. The lower values across the scene indicate a higher level of alignment between the two techniques when modeling simpler and more structured items, such as desks or electronic devices. Finally, Scene (C) yielded the smallest Hausdorff distances overall. All values remained below 0.3, and in several cases (e.g., meeting chair and cabinet) even below 0.1. This confirms that for geometrically simpler objects and more accessible angles, the differences between the two techniques become less pronounced.

These results support the fact that Polycam and LumaAI’s geometric similarity is object-dependent. For small or regular-shaped objects, the difference is less pronounced, while for more articulated or partially occluded geometries, LumaAI tends to diverge significantly, possibly due to overfitting surface detail.

Summary of the results

The analysis of the experimental results examined the performance of each technique from multiple perspectives. First, photogrammetry tools accelerated the reconstruction process by approximately two-thirds compared to manual modeling, significantly reducing completion time. However, manual modeling still achieved a lower polygon count and cleaner mesh topology, making it more suitable for real-time 3D applications. The SSIM values indicated no technique consistently achieved a significantly higher realism score than the original scene. The most notable differences emerged during real-time simulations, where the more complex models generated by LumaAI resulted in poorer performance. Finally, the Hausdorff distance analysis between LumaAI and Polycam models revealed that the two tools tended to produce substantially different mesh topologies from the same input video, highlighting that the choice of tool is highly dependent on the specific context and the characteristics of the object to be reconstructed.

5 Discussion

This section discusses some key points that emerged after analyzing the metrics results. First, we argue the main differences among the techniques and what influences them the most. Then, we provide a more in-depth comparison between the two tools. Finally, we highlight future work that could build upon our findings.

As a first overall observation, the results of this experiment confirmed and strengthened the findings reported in the previous study. As anticipated, the primary trade-off observed lies between processing time and the real-time usability of the reconstructed models. Manual modeling, which involves the direct intervention of a human expert to construct and optimize each object mesh, remains the most effective technique in terms of real-time performance and is not likely to be entirely replaced with the current technologies. This approach enables complete control over the structure and cleanliness of the mesh, resulting in lightweight and topologically efficient models. However, it also requires significant time investment and the involvement of personnel with both technical and artistic expertise. In contrast, mobile photogrammetry tools offer a more accessible and efficient alternative. These solutions significantly accelerate the modeling pipeline and require minimal expertise, making them suitable for rapid prototyping or use by non-expert users. Nevertheless, the models they produce tend to lack control over mesh topology, include possible reconstruction flaws, and often result in geometries with high polygon counts. Usually, in open-source photogrammetric pipelines, mesh density can be somewhat controlled by adjusting explicit software parameters. However, because of the adoption of ready-to-use mobile software, these adjustments

operate within architectural black boxes and the user has limited control on the output. Specifically, LumaAI implements a completely automated, non-configurable optimization pipeline, and Polycam restricts user to choose among generalized high, medium, and low quality setting. To manage this constraint within our experimental design, our protocol standardized all Polycam captures on the medium detail preset as a deliberate operational compromise between visual fidelity and real-time engine usability, while relying entirely on the standard automated delivery of LumaAI. For both tools, the output resultend in consistently highly polygonal meshes. Such results increase the computational load and negatively impact performance in real-time environments, particularly in hardware-constrained platforms such as mobile or VR systems.

✧ **The performance of reconstruction techniques was highly dependent on the characteristics of the individual objects.** Three main dimensions emerged as critical factors: object size, surface properties, and geometric complexity. First, object size had a notable impact on photogrammetry-based methods. Large objects posed difficulties during the acquisition phase, as capturing all relevant angles often required more physical space and movement, increasing the likelihood of occlusions or incomplete coverage. This limitation affected the completeness and consistency of the resulting meshes. Although limited viewpoints for larger objects also challenged manual modeling, the human modeler could mentally reconstruct occluded or unseen parts of the geometry, mitigating this issue to a certain extent. Second, reflective surfaces introduced significant challenges for both image- and video-based reconstruction. Since reflections vary across viewing angles, they generate inconsistencies across frames, leading to texture artifacts, geometric inaccuracies, or even holes in the reconstructed mesh. These artifacts were recurrent in objects with glossy finishes, where photogrammetry struggled to produce stable reconstructions. In contrast, manual modeling allowed the designer to ignore or reinterpret reflection effects, producing more stable and uniform geometry. Third, the geometric nature of the objects played a crucial role. Surprisingly, flat and straightforward objects—such as tables or wardrobes—proved problematic for photogrammetry algorithms. These approaches leverage light and shading variations to guide the reconstruction process, which is less effective when the surface lacks distinct features. Manual modeling, instead, was much more efficient in these cases, as simple shapes could be quickly assembled using basic primitives such as cubes or cylinders. Conversely, small and highly detailed objects—including plants, telephones, and keyboards—proved well-suited for photogrammetry. Their compact size facilitated full-angle coverage during scanning, and their intricate geometries provided abundant visual cues (e.g., edges, shadows, textures) that supported accurate reconstruction. Manual modeling, in contrast, would require substantially more time and effort to replicate the same level of detail, making it a less practical option for such objects. Figure 6 provides a visual summary of these observations, illustrating how different object features influenced the strengths and weaknesses of each reconstruction technique.

✧ **Can the photogrammetry tools replace the manual modeling? Short answer is no.** The short answer is no. Although photogrammetry-based tools offer significant advantages in terms of speed, ease of use, and accessibility, they cannot fully replace manual modeling for many professional applications. Photogrammetry

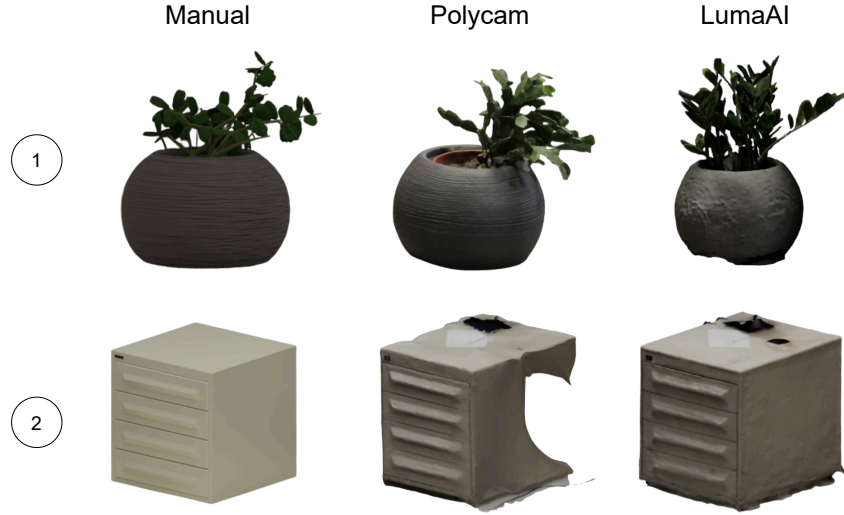


Fig. 6 Comparison of two models generated using the three reconstruction techniques. Row 1 shows a plant, a complex and highly detailed object that appears more realistic when reconstructed with photogrammetry tools, while the manually modeled version appears less convincing. Row 2 shows a cabinet, a simple geometric object, where the manual modeling produced a clean and accurate result, whereas the photogrammetry-based reconstructions, particularly with Polycam, exhibited more distortions.

dramatically accelerates the 3D reconstruction process and allows non-expert users to use it; however, it lacks the precision and control over mesh topology that manual modeling provides. The models generated through photogrammetry often result in dense and irregular meshes, making them unsuitable for real-time applications without substantial post-processing. Moreover, as previously highlighted, an important downside is represented by photogrammetry’s struggle with certain surface types—such as reflective, transparent, or featureless materials—and its limited resolution of missing or occluded parts of an object. Manual modeling remains indispensable in scenarios that require high-quality, optimized meshes for performance-sensitive environments such as VR, mobile platforms, and real-time simulations. It enables the creation of clean, low-polygon structures tailored to the application’s specific needs and allows creative adjustments beyond what is captured from reality. Additionally, manual intervention is often necessary to correct artifacts, refine textures, and optimize UV mapping, all of which are critical for professional-grade 3D assets. However, our findings confirm that rather than serving as a replacement, **photogrammetry tools should be regarded as complementary technologies.** They are particularly valuable for rapid prototyping, scanning complex organic objects, or generating assets for applications with less critical performance constraints. However, for scenarios demanding precision, optimization, and creative control, we argue that with the current technologies, manual expertise remains irreplaceable.

✍️ **Which of the analyzed tool is the best choice?** Having assessed that manual effort is still needed, the comparison of the two tools’ results gave us insights

into which one is more suitable for specific scenarios. Indeed, both tools exhibited specific strengths and weaknesses that varied depending on the type of object and surface characteristics related to their underlying technologies. As a first consideration, LumaAI produced higher-fidelity meshes than Polycam, being able to generate complete reconstruction most of the time. With the same input video, Polycam often generates incomplete meshes with holes or distortion. However, the models generated by LumaAI were consistently heavier in terms of polygon count—with no control over such an output—resulting in dense and computationally demanding geometries that negatively impacted real-time performance. Polycam, in contrast, produced lighter models that were more compatible with hardware-constrained platforms, even though this sometimes came at the expense of reconstruction completeness. When analyzing the tools’ performances, object type plays a significant role in tool performance. Polycam struggled with objects exhibiting mixed surface types—such as desks combining large flat surfaces with intricate details—leading to incomplete reconstructions and the presence of holes in the mesh. In these scenarios, LumaAI performed better, capturing both broad shapes and fine details more reliably, albeit with a heavier and more complex resulting mesh. Conversely, reflective surfaces represented a significant challenge for both methods, but Polycam proved to be more robust. For instance, when reconstructing objects like the whiteboard, Polycam managed to produce a distorted yet complete model, whereas LumaAI failed to deliver a stable or coherent reconstruction. Figure 7 shows an example of this comparison. In summary, the choice between the two tools should be guided by (1) the specific requirements of the target application, balancing fidelity against performance needs, and (2) the object’s characteristics and materials.

6 Conclusion & Future Work

This study investigated the practical applicability of ready-to-use mobile 3D capture applications for metaverse scene creation by conducting a comparative evaluation between traditional manual 3D modeling and two automated mobile tools (Polycam, leveraging an MVS pipeline, and LumaAI, leveraging a NeRF pipeline). The results confirmed that while automated mobile tools that exploit 3D reconstruction algorithm significantly accelerate the scene creation process—reducing the required time by more than two-thirds—they do not consistently outperform manual modeling in terms of polygonal optimization, rendering efficiency, or overall visual fidelity. Manual modeling remains the superior technique for producing clean, lightweight, and optimized assets suitable for real-time environments, particularly where resource constraints are critical. Conversely, photogrammetry tools offer an accessible and efficient alternative for rapid prototyping and for reconstructing complex objects, particularly when time or modeling expertise is limited. The results further revealed that the choice between different photogrammetry tools must be informed by object characteristics: while LumaAI generates higher-fidelity meshes at the cost of performance, Polycam provides lighter and more real-time-friendly models but at the risk of incomplete reconstructions.

The main contributions of this study are summarized as follows:

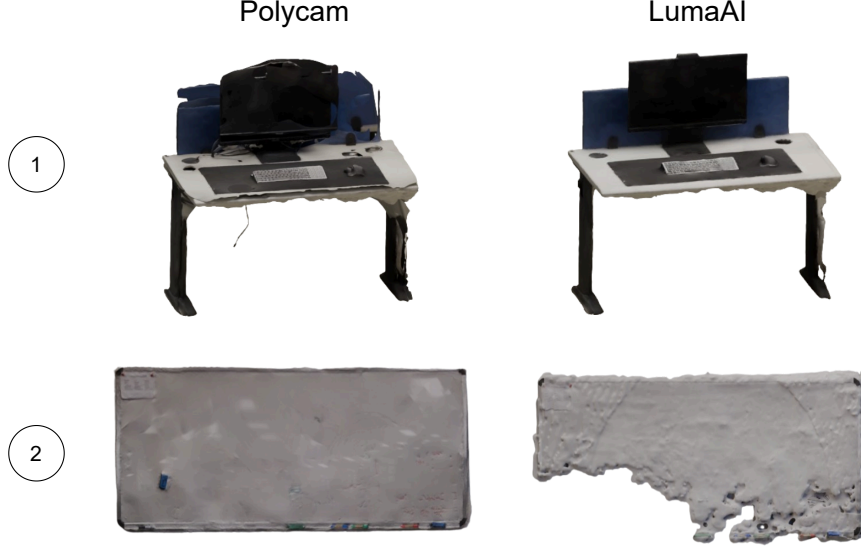


Fig. 7 Comparison of two models generated with Polycam and LumaAI. Row 1 shows the working desk, a case where LumaAI outperformed Polycam. This suggests that in the presence of heterogeneous surfaces with varying levels of detail, Polycam’s reconstruction tends to introduce inaccuracies, whereas LumaAI handles such complexity more effectively. Row 2 presents the whiteboard, where Polycam achieved a better result than LumaAI. This indicates that smooth, glossy, and reflective surfaces pose significant challenges for LumaAI, while Polycam demonstrates greater robustness in handling these material properties.

- A comparative analysis between manual modeling and for metaverse-oriented 3D scene creation.
- The identification of key trade-offs between visual fidelity, real-time performance, and modeling time across different workflow outcomes.
- Empirical guidelines supporting practitioners and researchers in selecting appropriate workflow based on project constraints and application scenarios.

This work opens several avenues for future research. First, the adoption of lightweight NeRF variants or hybrid optimization pipelines—such as Gaussian Splatting techniques—could be explored to address the performance limitations associated with current dense photogrammetric outputs, while also extending this investigation with novel practical recommendations. Second, the integration of post-processing steps, including mesh simplification, texture retargeting, and artifact correction, could be assessed to improve the real-time applicability of photogrammetry-derived models. Third, empirical studies evaluating user experience within metaverse environments reconstructed through different modeling techniques could reinforce these findings by incorporating human-centered evaluation metrics, such as perceived realism, immersion, and interaction quality. Finally, large-scale replications focusing on increasing the number of artists involved in the reconstruction process—rather than solely the

end users of the developed models and scenes—could generate new insights, further validating or refining the results presented in this work.

References

- [1] Stephenson, N.: Snow Crash. Bantam Books, New York, NY, USA (1992)
- [2] Ritterbusch, G.D., Teichmann, M.R.: Defining the metaverse: A systematic literature review. *Ieee Access* **11**, 12368–12377 (2023)
- [3] Pentangelo, V., Gravino, C., Palomba, F.: From zero to hero: a scoping review of the emergence of the metaverse in the virtual environments history. *Virtual Reality* (2026) <https://doi.org/10.1007/s10055-026-01340-3>
- [4] Dionisio, J.D.N., Burns, W.G., Gilbert, R.: 3d virtual worlds and the metaverse: Current status and future possibilities. *ACM Computing Surveys (CSUR)* **45**(3), 1–38 (2013)
- [5] Park, S.-M., Kim, Y.-G.: A metaverse: Taxonomy, components, applications, and open challenges. *IEEE access* **10**, 4209–4251 (2022)
- [6] Wang, H., Ning, H., Lin, Y., Wang, W., Dhelim, S., Farha, F., Ding, J., Daneshmand, M.: A survey on the metaverse: The state-of-the-art, technologies, applications, and challenges. *IEEE Internet of Things Journal* **10**(16), 14671–14688 (2023) <https://doi.org/10.1109/JIOT.2023.3278329>
- [7] Zhang, X., Chen, Y., Hu, L., Wang, Y.: The metaverse in education: Definition, framework, features, potential applications, challenges, and future research topics. *Frontiers in Psychology* **13**, 1016300 (2022)
- [8] Falahi, A.A., Alnuaimi, H., Alqaydi, M., Shateri, N.A., Ameri, S.A., Qbea’h, M., Alrabaee, S.: From challenges to future directions: A metaverse analysis. In: 2024 2nd International Conference on Intelligent Metaverse Technologies & Applications (iMETA), pp. 34–43 (2024). <https://doi.org/10.1109/iMETA62882.2024.10807918>
- [9] Mitra, S.: Metaverse: A potential virtual-physical ecosystem for innovative blended education and training. *Journal of Metaverse* **3**(1), 66–72 (2023)
- [10] Popov, V., Mateju, N., Jeske, C., Lewis, K.O.: Metaverse-based simulation: a scoping review of charting medical education over the last two decades in the lens of the ‘marvelous medical education machine’. *Annals of medicine* **56**(1), 2424450 (2024)
- [11] Popescu, G.H., Ciurlău, C.F., Stan, C.I., Băcănoiu, C., Tănase, A.: Virtual workplaces in the metaverse: Immersive remote collaboration tools, behavioral predictive analytics, and extended reality technologies. *Psychosociological Issues*

- in *Human Resource Management* **10**(1), 21–34 (2022)
- [12] Hale, K.S., Stanney, K.M.: *Handbook of Virtual Environments: Design, Implementation, and Applications*. CRC Press, USA (2014)
 - [13] Wimmer, M., Wonka, P.: Rendering time estimation for real-time rendering. In: *Rendering Techniques*, pp. 118–129 (2003)
 - [14] Lu, J., Li, C., Yin, C., Ma, L.: A new framework for automatic 3d scene construction from text description. In: *2010 IEEE International Conference on Progress in Informatics and Computing*, vol. 2, pp. 964–968 (2010). <https://doi.org/10.1109/PIC.2010.5687870>
 - [15] Cannavò, A., Demartini, C., Morra, L., Lamberti, F.: Immersive virtual reality-based interfaces for character animation. *IEEE Access* **7**, 125463–125480 (2019) <https://doi.org/10.1109/ACCESS.2019.2939427>
 - [16] Baqersad, J., Poozesh, P., Niezrecki, C., Avitabile, P.: Photogrammetry and optical methods in structural dynamics—a review. *Mechanical Systems and Signal Processing* **86**, 17–34 (2017)
 - [17] Cannavò, A., D’Alessandro, A., Maglione, D., Marullo, G., Zhang, C., Lamberti, F., *et al.*: Automatic generation of affective 3d virtual environments from 2d images. In: *Proc. 15th International Conference on Computer Graphics Theory and Applications (GRAPP 2020)*, pp. 113–124 (2020). SCITEPRESS
 - [18] Wang, X., Wang, C., Liu, B., Zhou, X., Zhang, L., Zheng, J., Bai, X.: Multi-view stereo in the deep learning era: A comprehensive review. *Displays* **70**, 102102 (2021)
 - [19] Mildenhall, B., Srinivasan, P.P., Tancik, M., Barron, J.T., Ramamoorthi, R., Ng, R.: Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM* **65**(1), 99–106 (2021)
 - [20] Remondino, F., Karami, A., Yan, Z., Mazzacca, G., Rigon, S., Qin, R.: A critical analysis of nerf-based 3d reconstruction. *Remote Sensing* **15**(14), 3585 (2023)
 - [21] Polycam: Using the Polycam App to Create 3D Models. Accessed: 2025-03-28. <https://learn.poly.cam/hc/en-us/sections/30298203696788-Using-the-Polycam-App-to-Create-3D-Models>
 - [22] Luma AI: Luma AI – Bringing 3D to Everyone. Accessed: 2025-03-28. <https://lumalabs.ai>
 - [23] Geris, A., Cukurbasi, B., Kilinc, M., Teke, O.: Balancing performance and comfort in virtual reality: A study of fps, latency, and batch values. *Software: Practice and Experience* **54**(12), 2336–2348 (2024)

- [24] Abramov, N., Lankegowda, H., Liu, S., Barazzetti, L., Beltracchi, C., Ruttico, P.: Implementing immersive worlds for metaverse-based participatory design through photogrammetry and blockchain. *ISPRS International Journal of Geo-Information* **13**(6), 211 (2024)
- [25] Gledhill, D., Novak, M.: A novel methodology for the optimization of photogrammetry data of physical objects for use in metaverse virtual environments. In: 2023 IEEE International Conference on Metrology for eXtended Reality, Artificial Intelligence and Neural Engineering (MetroXRaine), pp. 40–45 (2023). IEEE
- [26] Prasetyadi, A., Rezaldi, M.Y., Trianggoro, C., Febriandirza, A., Suhud, R., Gunawan, A., Ramdhan, H.M.: Digital avatar sub-metaverse modeling using terrestrial photogrammetry techniques. In: 2023 International Conference on Computer, Control, Informatics and Its Applications (IC3INA), pp. 221–225 (2023). IEEE
- [27] Pentangelo, V., Dario, D.D., Martino, V.D., Buono, M.D., Lambiase, S.: Accelerating 3d scene development for the metaverse: Lessons from photogrammetry and manual modeling. In: 2024 2nd International Conference on Intelligent Metaverse Technologies & Applications (iMETA), pp. 190–197 (2024). <https://doi.org/10.1109/iMETA62882.2024.10808008>
- [28] Ritterbusch, G.D., Teichmann, M.R.: Defining the metaverse: A systematic literature review. *IEEE Access* (2023)
- [29] Mystakidis, S.: Metaverse. *Encyclopedia* **2**(1), 486–497 (2022)
- [30] Aloqaily, M., Bouachir, O., Karray, F., Al Ridhawi, I., El Saddik, A.: Integrating digital twin and advanced intelligent technologies to realize the metaverse. *IEEE Consumer Electronics Magazine* **12**(6), 47–55 (2022)
- [31] Blender HQ Amsterdam: Blender. <https://www.blender.org> [Accessed: May 2024]
- [32] Baqersad, J., Poozesh, P., Niezrecki, C., Avitabile, P.: Photogrammetry and optical methods in structural dynamics—a review. *Mechanical Systems and Signal Processing* **86**, 17–34 (2017)
- [33] Wang, X., Wang, C., Liu, B., Zhou, X., Zhang, L., Zheng, J., Bai, X.: Multi-view stereo in the deep learning era: A comprehensive review. *Displays* **70**, 102102 (2021)
- [34] Abramov, N., Lankegowda, H., Liu, S., Barazzetti, L., Beltracchi, C., Ruttico, P.: Metamorphosis: A digital approach to transforming communities through photogrammetry and metaverse. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences* **48**, 1–8 (2024)

- [35] Murtiyoso, A., Markiewicz, J., Karwel, A., Kot, P.: Investigation on the use of nerf for heritage 3d dense reconstruction for interior spaces. *International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences* **48**, 115–121 (2023)
- [36] Croce, V., Billi, D., Caroti, G., Piemonte, A., De Luca, L., Véron, P.: Comparative assessment of neural radiance fields and photogrammetry in digital heritage: impact of varying image conditions on 3d reconstruction. *Remote Sensing* **16**(2), 301 (2024)
- [37] Wu, T., Yuan, Y.-J., Zhang, L.-X., Yang, J., Cao, Y.-P., Yan, L.-Q., Gao, L.: Recent advances in 3d gaussian splatting. *Computational Visual Media* **10**(4), 613–642 (2024)
- [38] Wang, Z., Bovik, A.C., Sheikh, H.R., Simoncelli, E.P.: Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing* **13**(4), 600–612 (2004)
- [39] Aspert, N., Santa-Cruz, D., Ebrahimi, T.: Mesh: Measuring errors between surfaces using the hausdorff distance. In: *Proceedings. IEEE International Conference on Multimedia and Expo*, vol. 1, pp. 705–708 (2002). IEEE
- [40] Pentangelo, V., Di Dario, D., De Martino, V., Dello Buono, M., Lambiase, S.: Towards AI-Powered Automatic 3D Scene Generation for the Metaverse: A Comparative Analysis of Manual and Photogrammetry Techniques. <https://figshare.com/s/c5f34288ebfc559eeebd>

Statements and Declarations

- Funding: The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.
- Competing interests: The authors have no relevant financial or non-financial interests to disclose.
- Author contribution: All authors contributed to the study conception and design. Material preparation, data collection and analysis were performed by Viviana Pentangelo and Marco Dello Buono. The first draft of the manuscript was written by Viviana Pentangelo and Stefano Lambiase, and all authors commented on previous versions of the manuscript. All authors read and approved the final manuscript.
- Data availability: The datasets generated and analyzed during the current study are available in the Figshare repository available at the following link: <https://figshare.com/s/c5f34288ebfc559eeebd>.